

贪心

贪心是信息学竞赛中一种很重要的解题方法，他不仅仅是一些题目的正确解法，还可能是一道题目中的部分算法，甚至还可以以错误的算法“骗得”更多的分数。在信息学竞赛中，很多题目往往要求最优解，这就和我们贪心的思维不谋而合，虽然很多时候贪心并不是正解，或者贪心的效率比较低，但是在我们不会做这道题的时候却能带给我们正确的思路。虽然他只是一种基本的算法，但是他的作用不逊于很多高级算法。

那么贪心究竟是什么呢？并没有标准答案，在每个人心中，贪心的定义是不同的，思想不同，算法也不同。但是他们都有一个共同的目的——将思维简单化

区间覆盖

第一类常见的问题是——区间覆盖问题，即给定一个完整的区间，再给定多段小区间，问关于覆盖的问题：

1：用这些小区间去覆盖这一段完整的区间，如果区间不可以重复，可以缺失，最多可以选多少个小区间。

2：用这些小区间去覆盖这一段完整的区间，如果区间可以重复，不可以缺失，最少需要选多少个小区间。

3：这些小区间加起来一共可以覆盖多长的区域，重复覆盖的地方只算一次，中间没覆盖的地方不算长度。

4：有多个教室，多个活动，所有活动都要举办，至少需要多少间教室。

种树

题目描述

某条街被划为 n 条路段，这 n 条路段依次编号为 $1 \dots n$ 。每个路段最多可以种一棵树。现在居民们给出了 h 组建议，每组建议包含三个整数 b, e, t ，表示居民希望在路段 b 到 e 之间至少要种 t 棵树。这些建议所给路段的区间可以交叉。请问：如果要满足所有居民的建议，至少要种多少棵树。

输入格式

第一行为 n ，表示路段数。

第二行为 h ，表示建议数。

下面 h 行描述一条建议： b, e, t ，用一个空格分隔。

输出格式

输出只有一个数，为满足所有居民的建议，所需要种树的最少数量。

解题思路

贪心策略：

区间固定，要求固定，要让种的树数量最少，那么就要尽可能多的把树种在区间重叠区域。

- 1、按区间终点从小到大排序
- 2、先统计区间已经种树的数量，再从末尾向前栽剩下的树。注意已经栽过树的位置要忽略

智力大冲浪

题目描述

小伟报名参加中央电视台的智力大冲浪节目。本次挑战赛吸引了众多参赛者，主持人为了表彰大家的勇气，先奖励每个参赛者 m 元。先不要太高兴！因为这些钱还不一定都是你的？！接下来主持人宣布了比赛规则：

首先，比赛时间分为 n 个时段，它又给出了很多小游戏，每个小游戏都必须在规定期限 t_i 前完成。如果一个游戏没能在规定期限前完成，则要从奖励费 m 元中扣去一部分钱 w_i ， w_i 为自然数，不同的游戏扣去的钱是不一样的。当然，每个游戏本身都很简单，保证每个参赛者都能在一个时段内完成，而且都必须从整时段开始。主持人只是想考考每个参赛者如何安排组织自己做游戏的顺序。作为参赛者，小伟很想赢得冠军，当然更想赢取最多的钱！注意：比赛绝对不会让参赛者赔钱！

输入格式

输入共四行。

第一行为 m ，表示一开始奖励给每位参赛者的钱；

第二行为 n ，表示有 n 个小游戏；

第三行有 n 个数，分别表示游戏 1 到 n 的规定完成期限；

第四行有 n 个数，分别表示游戏 1 到 n 不能在规定期限前完成的扣款数。

输出格式

输出仅一行，表示小伟能赢取最多的钱。

解题思路

贪心策略：每个任务花费时间相同，惩罚不同，很明显先去做贵的，每个任务都有一个完成期限，为了让前面可以做更多的任务，在最后期限完成即可。

- 1、按照惩罚值从大到小排序。

- 2、依次枚举这些任务，在其最后期限完成，如果最后期限已经被占用了，就向前移动，直到找到空闲时间为止。

可以考虑用线段树优化来快速找到插入的空位



总结

对于区间覆盖这一类问题，一般情况下都是根据起点或者终点来排序，然后每次贪心的选择最优解从而得到最终答案。



建筑抢修

题目描述

展开

小刚在玩JSOI提供的一个称之为“建筑抢修”的电脑游戏：经过了一场激烈的战斗，T部落消灭了所有Z部落的入侵者。但是T部落的基地里已经有N个建筑设施受到了严重的损伤，如果不尽快修复的话，这些建筑设施将会完全毁坏。现在的情况是：T部落基地里只有一个修理工人，虽然他能瞬间到达任何一个建筑，但是修复每个建筑都需要一定的时间。同时，修理工人修理完一个建筑才能修理下一个建筑，不能同时修理多个建筑。如果某个建筑在一段时间之内没有完全修理完毕，这个建筑就报废了。你的任务是帮小刚合理的制订一个修理顺序，以抢修尽可能多的建筑。

输入格式

第一行是一个整数N，接下来N行每行两个整数T1,T2描述一个建筑：修理这个建筑需要T1秒，如果在T2秒之内还没有修理完成，这个建筑就报废了。

输出格式

输出一个整数S，表示最多可以抢修S个建筑。

解题思路

贪心策略：如果让你去来选择维修顺序，你该如何选择呢？优先去选择截止时间最早的先完成，那么该题不就是和前面的线段覆盖完全一样吗？实际上并不是这样，因为只是说了该任务要在截止时间之前完成，并不是刚好在截止时间完成，很容易举出反例。

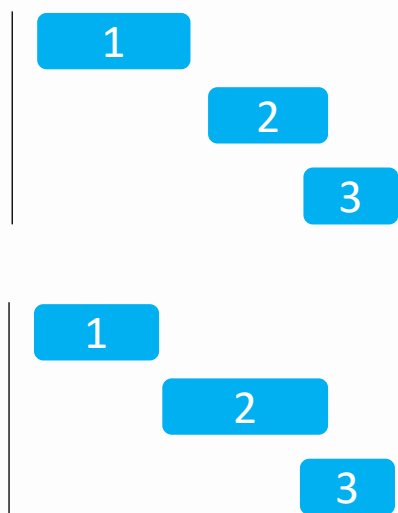


那么如果我们按照维修时间来贪心呢？优先去修维修时间最短的，这样贪心也有问题，很容易举出反例，因为我们是尽量在截止时间前修好，而不是先修。



解题思路

前面我们直接排序的贪心策略都是错误的，我们这里就要换一种贪心思想——在贪心的基础上尝试和替换，每次我先选择一个当前最优解，如果后面有更优的答案，那么我们就用后面的解去替换他。



如果我们发现当前这个任务没有办法完成的时候，我们就可以考虑是否可以把前面一个任务替换掉，根据什么来替换呢？维修时间，我们可以替换掉前面已经选择的一个维修时间最长的，这样既保证了前面的其他任务不会受到影响，也可以保证最后花的时间越短。

解题思路

- 1、先按照截止时间从小到大排序
- 2、对于当前任务，如果可以选择就先选择，如果该任务不可以完成，我们就去前面查找所有已经选择的任务，比较他们的维修时间大小，选出维修时间最大的一个，如果该任务维修时间比最大值小，那么替换，否则不可以完成。

因为涉及到每次找前面的最大值，并且涉及到插入删除操作，所以我们可以用优先队列优化。

总结

这一类带期限的贪心问题在竞赛也是很常见，问题模型为有很多件事情可以做，每件事情都有期限，问怎么做事才能保证做的事情最多。

这种题和前面的区间覆盖的题相比他的难度在于每件事情是有期限的，而没有固定的做的事情。这种情况一般用尝试和替换解决。先按照时间顺序排序，然后先在可以的情况下尝试做一件事情，如果后面发现有其他事前可以替换该事情(做完之后效果比当前更优)，就替换。一般为了保证时间复杂度，会用单调队列或者优先队列来维护最小值。



顺序选择问题

在贪心中还有一类问题是确定顺序的问题，该类问题和前面区间覆盖问题的区别在于所有物品(任务)都要选择，但是选择的顺序不同，答案也不相同，所以我们要确定一个正确的选择顺序，从而得到最优解。

这种题目如果能猜到顺序就直接猜，如果猜不到就可用数学表达式表不同顺序的答案表示出来，最后比较大小，或者说，如果要满足大小关系，必须要满足的条件，而这个条件就是排序的条件。



国王游戏

题目描述

[展开](#)

恰逢 H 国国庆，国王邀请 n 位大臣来玩一个有奖游戏。首先，他让每个大臣在左、右手上面分别写下一个整数，国王自己也在左、右手上各写一个整数。然后，让这 n 位大臣排成一排，国王站在队伍的最前面。排好队后，所有的大臣都会获得国王奖赏的若干金币，每位大臣获得的金币数分别是：排在该大臣前面的所有人的左手上的数的乘积除以他自己右手上的数，然后向下取整得到的结果。

国王不希望某一个大臣获得特别多的奖赏，所以他想请你帮他重新安排一下队伍的顺序，使得获得奖赏最多的大臣，所获奖赏尽可能的少。注意，国王的位置始终在队伍的最前面。

输入格式

第一行包含一个整数 n ，表示大臣的人数。

第二行包含两个整数 a 和 b ，之间用一个空格隔开，分别表示国王左手和右手上的整数。

接下来 n 行，每行包含两个整数 a 和 b ，之间用一个空格隔开，分别表示每个大臣左手和右手上的整数。

输出格式

一个整数，表示重新排列后的队伍中获奖赏最多的大臣所获得的金币数。

解题思路

假设前面所有人左手的乘积为 m ，后面两位大臣左右手的数分别是 (x_1, y_1) ， (x_2, y_2) ，那么：

如果第一位大臣在前面，答案为：

$$\max(m/y_1, m*x_1/y_2)$$

如果第二位大臣在前面，答案为：

$$\max(m/y_2, m*x_2/y_1)$$

因为 $m*x_1/y_2$ 一定大于 m/y_2 ， $m*x_2/y_1$ 一定大于 m/y_1 ，所以我们只需要考虑 $m*x_1/y_2$ 和 $m*x_2/y_1$ 的大小。

如果第一位大臣要在前面，那么就要满足 $x_1*y_1 < x_2*y_2$

所以排序的方式为左右手之积最小的排在前面，就能保证得钱最多的一位大臣的钱最少。



任务安排

有 n 个任务，每个任务都有一个最后期限时间 d_i ，还有一个完成需要的实际 t_i ，如果一个任务在时间 f_i 完成($f_i > d_i$)，那么该任务的延迟为 $f_i - d_i$ ，问，如何安排时间才能保证最大延迟最小。



解题思路

题目中有很明显的最大值最小，很容易想到二分答案，那么题目就变成了所有任务都必须要在 $d_i + ans$ 之前完成，判断是否可行。就和前面的裸的带期限的任务问题一样的做法。

那么该题是否可以直接贪心呢？我们不妨计算一下：

假设前面任务完成时间为 x ，后面两个任务分别为 (d_1, t_1) ， (d_2, t_2) ，那么两种情况最大延迟分别为：

$$\max(x + d_1 - t_1, x + d_1 + d_2 - t_2)$$

$$\max(x + d_2 - t_2, x + d_2 + d_1 - t_1)$$

只需要比较每一行最后一项的大小就可以了，最后得到，如果要想最大的延迟最小，那么 t 从小到大排序，和 d 完全没有关系。



打工

有 n 个工作，每个工作用 (a_i, b_i) 表示，令 E 为你的能力值，最开始为0，当你做完第 i 个工作后，能够得到 $E \cdot a_i$ 的钱，在得到钱之后你的能力值会增加 b_i ，那么你做下一个工作时你的能力值就是 $E + b_i$ 。问你最多能得到多少钱。



解题思路

该题和上一道题解题思路是一样的，没有办法只记得根据 a_i 或者 b_i 排序，所以我们考虑列出表达式：

第一个任务在前： $E \cdot a_1 + (E + b_1) \cdot a_2$

第二个任务在前： $E \cdot a_2 + (E + b_2) \cdot a_1$

很容易得到答案 $b_1 \cdot a_2 < b_2 \cdot a_1$

即根据 b_1/a_1 从小到大排序

饮料兑换

小明开了一家连锁店，连锁店共 n 家，出售饮料相同，现开展买饮料赠送活动。在第 i 家店，顾客可以用 a_i 个饮料瓶兑换到 b_i 瓶饮料和1个纪念币（不足 a_i 个不可以兑换），一家店可以兑换多次，兑换得到的饮料瓶还可以继续兑换。

问：小明买了 x 瓶饮料，最多可以得到多少枚纪念币。

输入：

5 2

5 3

2 1

输出：

4

饮料兑换

该题可以转换一下思路，变为考虑冷饮店的兑换顺序，又变为了一道顺序安排问题，很容易想到一种根据性价比来兑换饮料的方法，也很容易证明 $x/a_1 \cdot b_1 \geq x/a_2 \cdot b_2$ ，我们只需要选择 b_i/a_i 最大的就可以了。

但是实际上并不是这样，因为题目问的是最多的纪念币，而不是最多的饮料数。

如果整体考虑比较复杂，那么我们可以单独考虑，即考虑每一次兑换，每次只兑换 a_i 个，只要保证每次兑换最优，那么总体肯定是最优的。

分别在两个店兑换之后还剩下：

$$x - a_1 + b_1 \quad x - a_2 + b_2$$

剩下的纪念币越多，那么后面兑换的次数也就更多，所以我们应该按照 $b_1 - a_1$ 排序。

Barn Allocation G

农夫约翰最近开了一个新的牲口棚屋，并且现在接受来自奶牛的分配畜栏请求因为其中的一些畜栏有更好风景。

畜栏包括N个畜栏($1 \leq N \leq 100,000$)，方便起见，我们把它们编号为1..N，畜栏i能容纳 C_i 只牛($1 \leq C_i \leq 100,000$)，第i只牛需要连续编号畜栏 (从 A_i 到 B_i) 来漫步其中，

($1 \leq A_i \leq N$; $A_i \leq B_i \leq N$)，换言之，这只牛想要在编号范围为 $A_i..B_i$ 的畜栏漫步 (所有它想要畜栏必须实施为它空出位置来供它散步)

给出M个畜栏分配请求 ($1 \leq M \leq 100,000$)，回答最多能满足多少只牛的要求 (不增加另外畜栏)

考虑以下例子：

```
畜栏号:    1    2    3    4    5
          +---+---+---+---+---+
容纳空间:  | 1 | 3 | 2 | 1 | 3 |
          +---+---+---+---+---+

Cow 1      XXXXXXXXXXXX      (1, 3)
Cow 2              XXXXXXXXXXXX (2, 5)
Cow 3              XXXXXXXX   (2, 3)
Cow 4                      XXXXXXXX (4, 5)
```

约翰显然不能满足所有的牛，因为畜栏3, 4请求太多了

经过试验，我们发现，我们能满足牛1, 3, 4需要，所以这组数据答案为3

解题思路

该题和区间覆盖比较类似，对于这类问题，我们先考虑是根据起点还是终点排序，因为要满足更多的奶牛的要求，所以我们需要根据区间终点排序，终点相同的根据起点从大到小排序，因为这样占用的空间比较小。

对于枚举的每一个区间，判断这一段完整的区间是否满足条件，如果满足，答案就累加，如果不满足，就判断下一头奶牛。

但是该题数据比较大，一个一个去判断肯定过不了，再判断的时候本质上是判断区间最大值，所以我们可以用线段树来优化。

这是一道简单的线段树优化贪心的题。

Intervals

区间取数

题目描述

有 n 个区间，在区间 $[a_i, b_i]$ 中至少取任意互不相同的 c_i 个整数。求在满足 n 个区间的情况下，至少要取多少个正整数。

输入输出格式

输入格式

多组数据。

第一行的一个整数 T 表示数据个数。对于每组数据,第一行包含一个整数 $n(1 \leq n \leq 50000)$ 表示区间数。以下 n 行描述区间。输入的第 $(i+1)$ 行包含三个整数 a_i, b_i, c_i ，由空格分开。其中 $0 \leq a_i \leq b_i \leq 50000, 1 \leq c_i \leq b_i - a_i + 1$ 。

输出格式

对于每组数据，输出一个对于 n 个区间 $[a_i, b_i]$ 至少取 c_i 个不同整数的数的总个数。

解题思路

该题和种树那道题其实是一样的，只是该题数据范围比较大，达到了 10^5 ，所以暴力枚举插入的位置是过不了的，所以需要线段树优化。

先按照终点从小到大排序，然后线段树区间修改，对于一个区间，如果右子树区间空余结点数量 $\geq k$ ，那么直接更新右子树，如果 $< k$ ，那么说明在右子树填满的基础上还需要去左子树更新，直接把右子树打上一个标记就可以了。接下来重复前面操作。

这是一道不错的线段树练手题，和常规的线段树查询修改有一点区别。

当然，该题还可以用差分约束来做，区间 $[a, b]$ 至少种 c 棵树，相当于 $a - b \geq c$ 。

旅行家的预算

题目描述

展开

一个旅行家想驾驶汽车以最少的费用从一个城市到另一个城市（假设出发时油箱是空的）。给定两个城市之间的距离 $D1$ 、汽车油箱的容量 C （以升为单位）、每升汽油能行驶的距离 $D2$ 、出发点每升汽油价格 P 和沿途油站数 N （ N 可以为零），油站 i 离出发点的距离 Di 、每升汽油价格 Pi （ $i = 1, 2, \dots, N$ ）。计算结果四舍五入至小数点后两位。如果无法到达目的地，则输出“No Solution”。

输入格式

第一行， $D1, C, D2, P, N$ 。

接下来有 N 行。

第 $i + 1$ 行，两个数字，油站 i 离出发点的距离 Di 和每升汽油价格 Pi 。

输出格式

所需最小费用，计算结果四舍五入至小数点后两位。如果无法到达目的地，则输出“No Solution”。

解题思路

很容易想到一种贪心策略，就是优先加油价便宜的，但是由于有距离限制，所以我们也不知道前面应该加多少油。所以该题也是一种尝试+替代的思路。

假设邮箱可以加不同的油，并且可以优先选择价格最低的用，并且还可以退油，那么每到一个加油站，我们就可以考虑下应该怎么办？

1、如果油价比前面某些未用完的油低，那么前面这些未用完的油就不需要再用了，退回去，用该油去替代它。

2、如果该油比油箱里的所有油都贵怎么办呢？不加吗？并不是，因为我们要随时保持油箱都是满的，所以加满。

我们可以维护一个单调递增队列，每次加油在队尾操作，每次用油在队首操作。就大大降低了时间复杂度。

树的路径覆盖

树的路径覆盖 (cover.cpp)

【问题描述】

给出一棵含 n 个结点的树，求它的一个最小路径覆盖。路径覆盖是指将点集划分为若干点不相交的路径的方案。

【输入格式】

本题有多组数据。

第一行包含一个正整数 t ($t \leq 10$) 表示数据组数

对于每组数据，第一行一个整数 n ，表示树的结点个数

后面 $n-1$ 行两个整数 u, v 表示结点 u 和 v 相连

【输出格式】

对于每组数据，输出最小路径覆盖数，数据间用回车隔开

解题思路

该题是一道经典的树上贪心问题，当然树形dp也可以做，像这种树上贪心或者树形dp的题一般就两种做法——自上而下或者自下而上。

我们可以把树上路径覆盖看做一堆零散的点合并。

如果一个点只有一个儿子结点，那么，很明显，能够颜色到儿子结点的链一定可以继续延伸到其父亲结点。

但是如果一个点有多个儿子结点呢？那么就有两种可能，第一种是沿着父亲结点继续延伸，第二种是合并两条直链（直链的意思是没有向下完全的链）。那么哪一种更优呢？明显是合并两条链，因为如果不合并这两条链，那么另一条链一定会增加一条路径，但是向上的点可能被上面其他路径覆盖，路径长度本来就不会增加。



解题思路

所以该题的贪心策略就是对于一颗子树，如果其叶子节点所在链还可以合并（没有完全，是一条直连），那么就合并这两条链，并且其父亲节点在这条弯链上，打上标记。说明其父亲结点不可以再向上继续合并了。



Main Sequence

用一个长度为 n 的“(”或者“)”序列，问是否可以把其中的某些“(”变为“)”保证该式子括号匹配(只能把“(”变成“)”，不能反过来)

输入：

3

((((

()))

)(((

输出：

Yes

No

NO

解题思路

该题也比较简单，因为我们只能把“(”变成“)”，所以如果我们正着枚举，因为无法判断变还是不变，所以很麻烦，所以我们干脆倒着枚举，枚举到一个“)”，那肯定需要一个“(”来匹配，如果枚举到一个“(”，并且后面没有与之匹配的“)”，很明显，此处的“(”肯定要变成“)”，最后看下是否还有剩下的“(”或者“)”就可以了，和普通字符串模拟一样，用一个栈模拟就可以了。

Codeforces 286C : Main Sequence

总结

在竞赛中，很多时候需要逆向思维，就是当你按照题目给定的方法无法得到答案或者很难计算的时候，不妨倒过去去思考，比如树拆分成链，看做点合并成树；把摧毁看做合并；正着判断变为倒着判断；增加看做减少。很多时候会有意想不到的收获



必做练习题

LOJ 10002: 喷水装置

洛谷 4053: 建筑抢修

洛谷UVA1723 Intervals

洛谷 1016: 旅行家的预算

BZOJ 1907: 树的最小路径覆盖

